

PATHFINDER 2E

CENTRALITA DE CAMPAÑAS

Especificación profesional de producto, arquitectura y diseño



Una plataforma modular que combina gestor de campañas, gestor de personajes, bestiario y wiki/compendio en una única base de conocimiento reutilizable.

Objetivo central: crear una “centralita” para cualquier campaña de Pathfinder 2e, donde el contenido del mundo y las reglas se almacenen una sola vez y puedan reutilizarse en múltiples campañas.

1. Resumen ejecutivo

El producto será una aplicación web de gestión de campañas y conocimiento para Pathfinder 2e Remaster. No estará ligada a una sola aventura: funcionará como una base de datos personal de reglas, contenido y mundo, sobre la que se crean múltiples campañas independientes.

La idea clave es separar contenido reutilizable de datos específicos de una campaña. Una dote, criatura, objeto o hechizo puede existir en el compendio global y ser utilizado por varias campañas; una misión, sesión, secreto o nota de director pertenece a una campaña concreta.

La aplicación debe permitir tanto consultar información como crear y editar información desde la propia interfaz. El usuario no debería necesitar modificar archivos JSON ni tocar la base de datos para añadir una dote, un NPC, una localización, un hechizo o un objeto.

1.1 Objetivos del producto

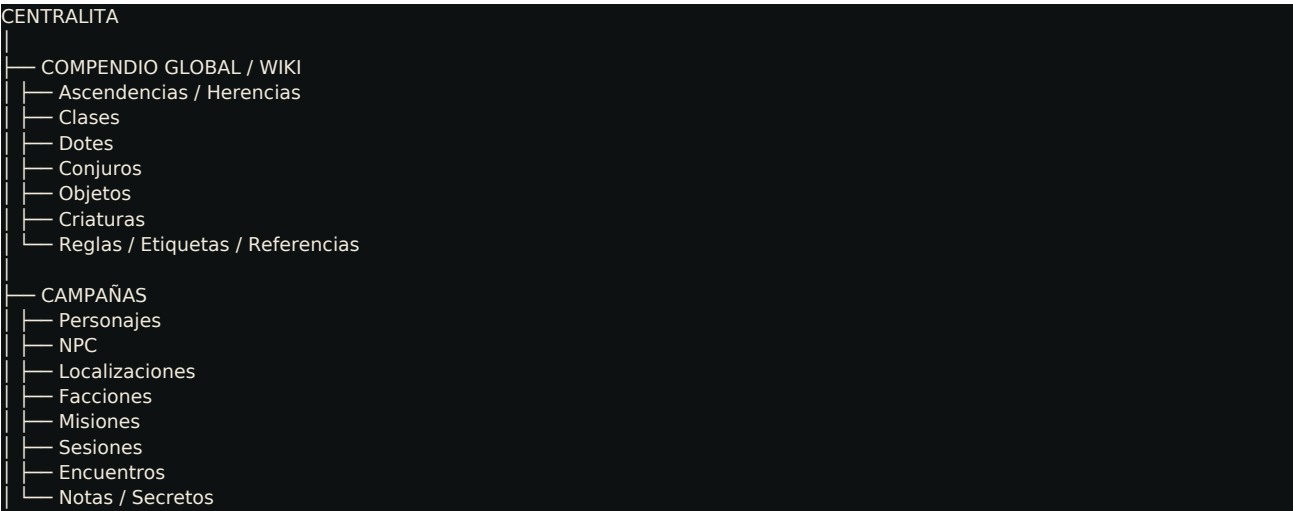
- Centralizar campañas, personajes, NPC, localizaciones, misiones, sesiones y notas.
- Crear un compendio reutilizable de ascendencias, clases, dotes, conjuros, objetos, criaturas y reglas.
- Permitir alta, edición, duplicación, archivado y organización del contenido desde la aplicación.
- Relacionar entidades mediante enlaces y referencias bidireccionales.
- Ofrecer una experiencia visual coherente con una wiki de fantasía medieval: oscuro, pergamino, dorado y rojo.
- Ser usable en escritorio y móvil mediante diseño responsive.
- Preparar la arquitectura para futuras ampliaciones sin rehacer el núcleo.

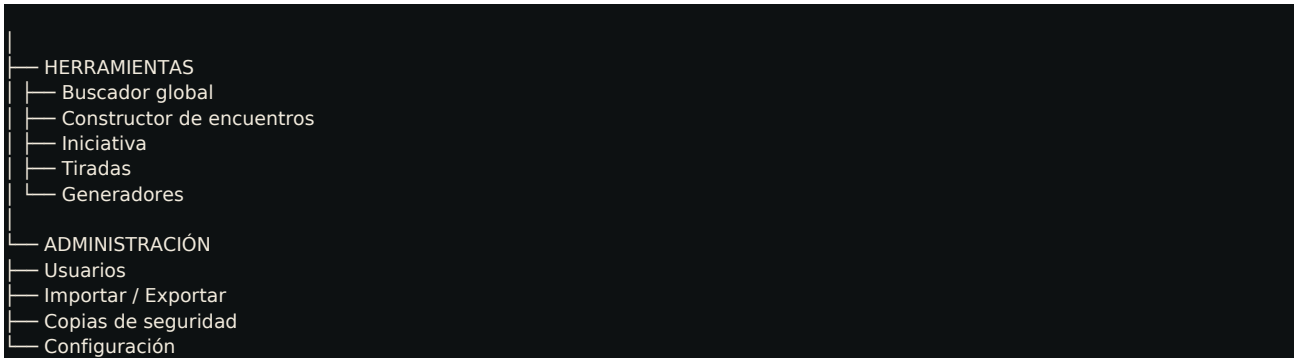
1.2 Principios de diseño

Principio	Aplicación
Modularidad	Cada dominio —campañas, personajes, compendio, bestiario, etc.— se desarrolla como módulo independiente.
Datos primero	La interfaz representa entidades y relaciones; no se codifica contenido directamente en componentes.
Reutilización	El mismo objeto, dote o criatura puede aparecer en varias campañas.
Editable	El usuario puede crear y modificar contenido desde la aplicación.
Trazabilidad	Cambios importantes pueden quedar registrados en historial/auditoría.
Escalabilidad progresiva	Se comienza como monolito modular y se incorporan servicios adicionales solo cuando sean necesarios.

2. Modelo conceptual del producto

La arquitectura funcional se organiza en cuatro capas conceptuales:





2.1 Regla fundamental: compendio vs campaña

Dato	Compendio global	Campaña
Dote	Sí	Referencia/selección
Conjuro	Sí	Referencia/selección
Objeto	Sí	Puede tener ejemplares concretos
Criatura	Sí	Puede tener instancia en encuentro
NPC	Plantilla opcional	Sí, normalmente específico
Localización	Puede existir como canon/general	Versión concreta de la campaña
Misión	No	Sí
Sesión	No	Sí
Secreto del GM	No	Sí

Ejemplo: “Ahuizotl” existe una vez en el bestiario. La Campaña A y la Campaña B pueden utilizarlo sin duplicar la ficha. Sin embargo, “Ahuizotl de la sesión 7” puede ser una instancia concreta con PV actuales, condiciones, notas y posición en combate.

3. Arquitectura de información y navegación

La navegación principal debe permanecer estable para que el usuario pueda cambiar rápidamente entre módulos.

Sección	Función principal
Inicio	Resumen de campañas, actividad reciente, accesos rápidos y búsqueda.
Campañas	Crear/seleccionar campaña y gestionar su contenido específico.
Personajes	Fichas de PJ, progresión, equipo, dotes, conjuros y notas.
NPC	Base de NPC con relaciones, ubicación, facción y apariciones.
Localizaciones	Wiki de lugares con mapa, descripción, habitantes y enlaces.
Misiones	Objetivos, estado, recompensas, NPC y localizaciones relacionadas.
Sesiones	Preparación, diario de sesión, escenas, encuentros y resultados.
Bestiario	Búsqueda y consulta de criaturas, filtros y favoritos.
Compendio	Ascendencias, clases, dotes, conjuros, objetos y reglas.
Herramientas	Iniciativa, encuentros, tiradas y utilidades.
Ajustes	Cuenta, tema, datos, copias, importación/exportación y permisos.

3.1 Página de entidad universal

Todas las entidades importantes deben compartir un patrón visual: breadcrumb, título, tipo, imagen opcional, resumen, pestañas y panel lateral de metadatos. Esto permitirá que la aplicación se sienta como una wiki coherente.

Breadcrumb → Título + Tipo + Favorito

Resumen / descripción

Pestañas: Información | Relaciones | Historial | Notas

Contenido principal Panel lateral

- etiquetas
- fuente
- campaña
- relaciones
- acciones

4. Dirección visual y sistema de diseño

La interfaz debe conservar el lenguaje visual del prototipo generado: fantasía editorial, sobria y funcional, evitando que el ornamento dificulte la lectura.

Uso	Color	Hex
Fondo principal	Carbón	#0D1112
Panel	Pizarra	#171B1C
Pergamino	Base	#E8D9B8
Dorado	Acento	#B8933F
Rojo	Acento/selección	#7A1F1F
Texto oscuro	Contenido	#2B1A12
Texto claro	Sobre fondo oscuro	#F2EBDD

4.1 Tipografía

Se recomienda una serif para títulos y una sans-serif para contenido, tablas, formularios y navegación. La prioridad es legibilidad. La tipografía exacta podrá sustituirse posteriormente por una familia con licencia adecuada si se desea una identidad más fantástica.

4.2 Componentes reutilizables

- Sidebar / navegación principal.
- Breadcrumbs.
- Cards de entidades.
- Ficha de entidad.
- Tabs.
- Badges de nivel, rasgo, tipo y estado.
- Buscador con filtros.
- Tablas ordenables.
- Editor de texto enriquecido.
- Selectores de entidades relacionadas.
- Modal de creación rápida.
- Panel lateral contextual.
- Timeline de sesiones/historial.
- Mapa interactivo.

- Toast/notificaciones y confirmaciones.

5. Módulos funcionales

Módulo	MVP	Futuro
Gestor de campañas	Campañas, resumen, sesiones, misiones, NPC y lugares	Calendario, estadísticas, automatizaciones
Gestor de personajes	Ficha, niveles, atributos, dotes, conjuros, equipo	Cálculos avanzados, subida de nivel guiada
Bestiario	CRUD, filtros, búsqueda, ficha de criatura	Encuentros, tácticas, IA asistida
Compendio	Ascendencias, clases, dotes, conjuros, objetos	Reglas cruzadas y recomendaciones contextuales
Wiki	Páginas enlazadas, categorías y relaciones	Grafo visual del mundo
Mapas	Imagen + marcadores	Mapas por capas y edición avanzada
Sesiones	Preparación y diario	Timeline, control de escenas y recap automático
Herramientas	Iniciativa y tiradas básicas	Constructor de encuentros y GM toolkit

6. Sistema de creación y edición de contenido

Este es un requisito estructural, no una función secundaria. El usuario debe poder crear contenido sin salir de la aplicación.

6.1 Flujo estándar de creación

Nuevo → seleccionar tipo de entidad → formulario → guardar
↓
validar datos y relaciones
↓
crear registro + historial
↓
disponible en búsquedas y enlaces

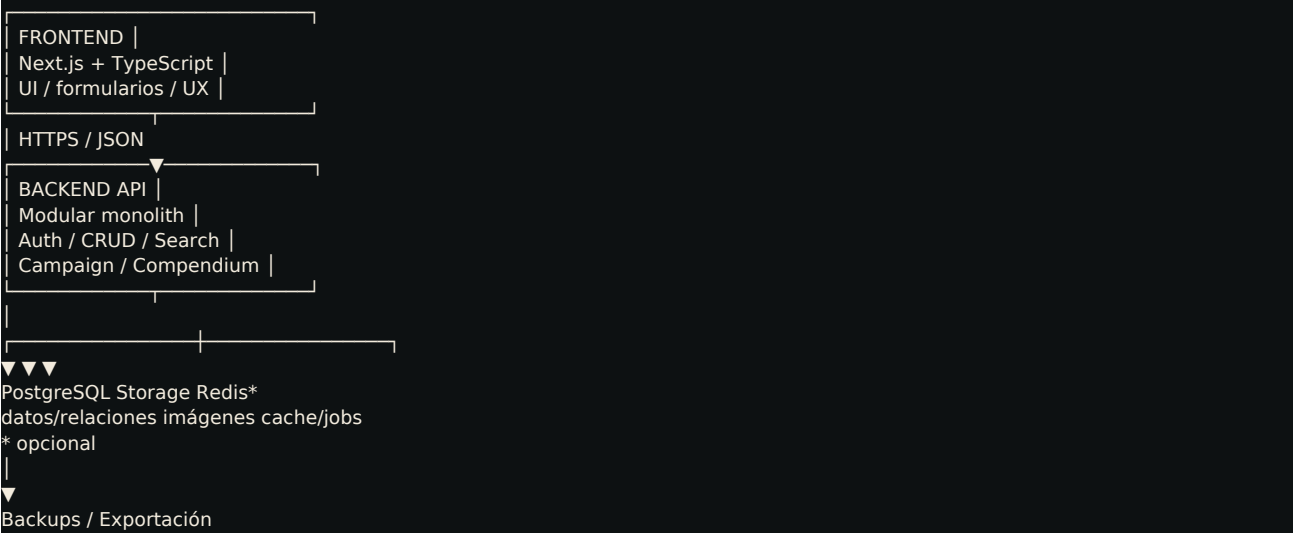
Entidad	Campos principales
Dote	Nombre, nivel, tipo, rasgos, requisitos, acciones, descripción, fuente, enlaces
Conjuro	Nombre, nivel, tradición, acciones, alcance, objetivos, duración, descripción, fuente
Objeto	Nombre, nivel, categoría, precio, Bulk, rasgos, activación, descripción, fuente
Localización	Nombre, tipo, región, descripción, población, mapa, coordenadas, NPC, relaciones
NPC	Nombre, rol, nivel, descripción, retrato, facción, localización, relaciones, notas
Personaje	Nombre, jugador, ascendencia, clase, nivel, atributos, PV, CA, dotes, equipo, conjuros
Criatura	Nombre, nivel, rasgos, estadísticas, sentidos, velocidades, ataques, habilidades, notas

6.2 Edición y versionado

- Guardar borradores antes de publicar cambios.
- Historial de modificaciones por entidad.
- Restauración de una versión anterior.
- Duplicar entidad para crear una variante.
- Archivar sin borrar definitivamente.
- Campos de fuente/origen para diferenciar contenido propio, importado o de referencia.
- Registro de quién realizó un cambio cuando exista colaboración.

7. Arquitectura técnica propuesta

Para una primera versión profesional se recomienda un monolito modular con API bien definida. Evita la complejidad prematura de microservicios, pero permite separar módulos posteriormente si el proyecto crece.



7.1 Stack recomendado

Capa	Tecnología propuesta	Motivo
Frontend	Next.js + React + TypeScript	Aplicación web moderna, routing, SSR/CSR y ecosistema amplio.
Estilos	CSS Modules/Tailwind + design system propio	Permite reproducir con precisión el tema visual.
Estado	TanStack Query + estado local	Separación clara entre datos del servidor y estado de UI.
Backend	Node.js + NestJS o API modular equivalente	Arquitectura organizada por dominios y validación.
ORM	Prisma	Tipado, migraciones y acceso estructurado a PostgreSQL.
Base de datos	PostgreSQL	Relaciones, búsquedas, integridad y crecimiento futuro.
Archivos	S3-compatible	Retratos, mapas, iconos y adjuntos.
Cache	Redis opcional	Solo cuando la carga lo justifique.
Testing	Vitest/Jest + Playwright	Pruebas unitarias, API y flujos completos.
Deploy	Docker + proveedor cloud	Entornos reproducibles y despliegue controlado.

8. Modelo de datos

La base de datos debe ser relacional y diseñada alrededor de entidades enlazables.

Entidad	Relaciones clave
User	Campaigns, characters, permissions, audit logs
Campaign	Characters, NPCs, locations, quests, sessions, encounters
Character	Campaign, ancestry, class, feats, spells, items, notes
NPC	Campaign, faction, location, quests, relationships
Location	Campaign/setting, parent location, NPCs, quests, map markers
Quest	Campaign, NPCs, locations, rewards, sessions
Session	Campaign, participants, quests, encounters, notes
Ancestry	Feats, traits, characters
Class	Feats, spells, characters

Entidad	Relaciones clave
Feat	Class/ancestry/general, prerequisites, traits, source
Spell	Traditions, traits, classes, source
Item	Traits, categories, characters, source
Creature	Traits, attacks, abilities, encounters, source
Tag	Many-to-many with most entities
Relation	Generic links between entities when appropriate
AuditLog	User, entity, action, timestamp, before/after metadata

8.1 Relaciones genéricas

Además de relaciones tipadas, conviene disponer de una capa de enlaces genéricos para representar relaciones narrativas: “conoce a”, “vive en”, “enemigo de”, “forma parte de”, “aparece en”, “menciona a”, etc. Esto permite que la aplicación evolucione hacia una auténtica wiki enlazada.

9. API y backend

La API debe organizarse por dominios, no por páginas de interfaz.

Dominio	Ejemplos de endpoints
Auth	POST /auth/login, POST /auth/refresh, POST /auth/logout
Campaigns	GET/POST /campaigns, GET/PATCH/DELETE /campaigns/:id
Characters	GET/POST /characters, PATCH /characters/:id
NPCs	GET/POST /npcs, PATCH /npcs/:id
Locations	GET/POST /locations, PATCH /locations/:id
Quests	GET/POST /quests, PATCH /quests/:id
Sessions	GET/POST /sessions, PATCH /sessions/:id
Compendium	GET/POST/PATCH /feats, /spells, /items, /creatures
Search	GET /search?q=...&type;=...&tag;=...
Relations	POST /relations, DELETE /relations/:id
Files	POST /files/presign, DELETE /files/:id
Import/Export	POST /imports, GET /exports/:id

9.1 Validación

- Validar datos en backend aunque el frontend ya los valide.
- Usar esquemas tipados para entrada/salida.
- Controlar permisos en cada operación.
- No confiar en IDs o roles enviados por el cliente.
- Registrar errores de forma estructurada.
- Evitar eliminar entidades con relaciones críticas sin confirmación o estrategia de archivado.

10. Buscador y navegación wiki

El buscador será una pieza central. Debe poder localizar una entidad independientemente de dónde se encuentre.

- Búsqueda por nombre.

- Filtros por tipo de entidad.
- Filtros por nivel, rasgos, categoría, campaña y etiquetas.
- Búsqueda dentro de descripciones y notas.
- Resultados agrupados por tipo.
- Favoritos y recientes.
- Enlaces internos entre entidades.
- Historial de navegación.
- Búsqueda rápida tipo Ctrl+K / búsqueda global.

Evolución recomendada: comenzar con índices y búsqueda de PostgreSQL. Incorporar un motor dedicado como Meilisearch/OpenSearch solo si el volumen o las necesidades de relevancia lo justifican.

11. Gestor de campañas

Cada campaña funciona como un espacio de trabajo que reutiliza el compendio global sin contaminarlo.

Pantalla	Contenido
Dashboard	Próxima sesión, misiones activas, personajes, eventos y actividad reciente.
Personajes	Lista de PJ, nivel, clase, estado y acceso a ficha.
NPC	NPC de la campaña, relaciones y apariciones.
Localizaciones	Mapa, lugares visitados y lugares pendientes.
Misiones	Activas, completadas, fallidas, ocultas y futuras.
Sesiones	Preparación, notas, encuentros y resumen.
Encuentros	Criaturas, iniciativa, dificultad, recompensas y resultado.
Notas	Información libre, secretos del GM y referencias.
Timeline	Eventos importantes de la campaña en orden temporal.

12. Gestor de personajes

La ficha debe poder utilizarse tanto para consultar al personaje como para mantenerlo actualizado durante la campaña.

- Resumen: nivel, PV, CA, percepción, atributos y estado.
- Ascendencia y herencia.
- Clase y características de clase.
- Dotes por nivel.
- Habilidades y competencias.
- Conjuros y recursos mágicos.
- Equipo, inventario y Bulk.
- Historial de progresión.
- Notas personales.
- Relación con campaña, jugador y NPC.

La arquitectura debe evitar calcular reglas complejas en múltiples lugares. Los cálculos derivados deben vivir en un servicio/regla central para que la ficha, el combate y otras pantallas utilicen la misma fuente.

13. Bestiario interactivo

El bestiario será simultáneamente una herramienta de consulta y una fuente para crear encuentros.

Función	Descripción
Búsqueda	Nombre y texto completo.
Filtros	Nivel, tipo, rasgos, entorno, rareza y etiquetas personalizadas.
Ficha	Estadísticas, sentidos, velocidades, habilidades, ataques y capacidades.
Favoritos	Acceso rápido a criaturas frecuentes.
Notas	Notas privadas sobre uso, tácticas o adaptación.
Añadir criatura	Formulario completo desde la propia aplicación.
Duplicar	Crear una variante para una campaña sin alterar la entrada base.
Encuentro	Enviar criatura a un encuentro o iniciativa.

14. Wiki y mapas

Las localizaciones deben poder comportarse como páginas de wiki y como nodos del mundo.

- Jerarquía: mundo → continente → región → país → ciudad → distrito → edificio → sala.
- Breadcrumbs automáticos.
- Enlaces a NPC, facciones, misiones y sesiones.
- Imagen principal y galería.
- Mapa con marcadores.
- Estado de conocimiento: público, GM, secreto.
- Relaciones entre lugares.
- Historial de cambios.

Futuro: mapa interactivo con capas activables, coordenadas, zoom y marcadores que abran fichas de entidades.

15. Usuarios, permisos y privacidad

Rol	Capacidades
Propietario	Control total de cuenta, campañas, datos y configuración.
GM/Director	Gestiona campaña y contenido privado.
Jugador	Consulta y edita únicamente lo permitido de su personaje.
Colaborador	Puede editar determinados módulos si se habilita colaboración.
Solo lectura	Consulta sin modificar.

Debe existir separación clara entre información visible para jugadores y secretos del GM. Esta separación debe ser una propiedad de permisos del backend, no simplemente una ocultación visual del frontend.

16. Importación, exportación y copias

- Exportación completa de una campaña.
- Exportación selectiva de entidades.
- Formato JSON versionado para portabilidad.
- Exportación de páginas a Markdown/HTML cuando sea útil.

- Importación de datos estructurados.
- Copias de seguridad periódicas.
- Restauración de campañas.
- Identificadores estables para evitar duplicados.

El formato de datos debe estar versionado: por ejemplo, schemaVersion: 1.0. Esto permitirá modificar la estructura interna sin romper las copias antiguas.

17. Seguridad y calidad

- Contraseñas gestionadas mediante un proveedor seguro o hashing moderno.
- Sesiones con expiración y renovación controlada.
- Autorización en backend por recurso.
- Validación de archivos subidos y límites de tamaño.
- Protección contra XSS en contenido enriquecido.
- Protección CSRF cuando aplique al mecanismo de autenticación.
- Rate limiting en autenticación y endpoints sensibles.
- Backups automatizados.
- Logs de errores sin almacenar secretos.
- Tests de regresión antes de cada despliegue.

18. Estructura del proyecto

```
centralita-pf2e/
├── apps/
│   ├── web/ # Frontend Next.js
│   └── api/ # Backend modular
├── packages/
│   ├── ui/ # Design system
│   ├── types/ # Tipos compartidos
│   ├── rules/ # Cálculos y lógica PF2e
│   └── config/ # Configuración común
├── prisma/
│   ├── schema.prisma
│   └── migrations/
├── docs/
│   ├── architecture/
│   ├── api/
│   └── product/
├── tests/
├── docker/
├── compose.dev.yml
└── compose.prod.yml
```

La estructura exacta puede simplificarse para el MVP. Lo importante es mantener una separación clara entre presentación, dominio, persistencia y datos.

19. Roadmap de desarrollo

Fase	Resultado
0 — Fundación	Repositorio, diseño visual, autenticación, PostgreSQL, migraciones, CI y componentes base.
1 — Centralita	Dashboard, campañas, navegación, buscador y CRUD genérico.
2 — Compendio	Ascendencias, clases, dotes, conjuros, objetos y bestiario.
3 — Personajes	Ficha completa, progresión, equipo, dotes y conjuros.

Fase	Resultado
4 — Campañas	NPC, lugares, misiones, sesiones, notas y relaciones.
5 — Herramientas	Iniciativa, encuentros, tiradas y utilidades.
6 — Mapas/Wiki	Mapas interactivos, grafo de relaciones y wiki avanzada.
7 — Colaboración	Invitaciones, roles, permisos y trabajo con jugadores.
8 — Automatización	Import/export avanzado, backups, plantillas y asistentes.

19.1 MVP recomendado

Para evitar que el proyecto se vuelva inabarcable, el primer objetivo funcional debe ser:

- Login y perfil.
- Dashboard.
- Crear múltiples campañas.
- Crear/editar/eliminar/archivar entidades.
- Compendio básico.
- Bestiario.
- Gestor de personajes.
- NPC y localizaciones.
- Buscador global.
- Relaciones entre entidades.
- Importación/exportación JSON.
- Tema visual definitivo.

20. Ampliaciones futuras

- Modo combate con iniciativa y condiciones.
- Constructor automático de encuentros.
- Generador de NPC y tesoros.
- Calendario interno de campaña.
- Sistema de cronología y eras.
- Árbol genealógico de personajes.
- Mapa mundial interactivo.
- Mapas de combate por cuadrícula.
- Seguimiento de reputación y facciones.
- Generación asistida de resúmenes de sesión.
- Búsqueda semántica sobre las notas de la campaña.
- Aplicación móvil/PWA.
- Modo offline con sincronización.
- Compartir campañas con jugadores.
- Plantillas de campañas.
- Versionado completo de mundos.

- Sistema de plugins/extensiones.

21. IA como capa futura, no como dependencia del núcleo

La IA puede añadirse después sin convertirla en requisito estructural. La base de datos debe ser útil por sí misma.

Función	Ejemplo
Consulta	“¿Qué NPC están relacionados con la Mansión DoRoux?”
Preparación	Generar un resumen de la próxima sesión a partir de notas.
Recap	Convertir notas de sesión en un resumen organizado.
Búsqueda	Encontrar referencias relacionadas aunque no coincida exactamente el nombre.
Ayuda al GM	Proponer encuentros basados en nivel y localización.
Mantenimiento	Detectar entidades duplicadas o relaciones incompletas.

22. Criterios de aceptación del producto

1. El usuario puede crear más de una campaña sin mezclar sus datos.
2. Una entidad del compendio puede reutilizarse en múltiples campañas.
3. El usuario puede crear y editar contenido desde la interfaz.
4. Todas las entidades relevantes pueden enlazarse entre sí.
5. El buscador puede localizar entidades por nombre y filtros.
6. Los cambios no rompen relaciones existentes.
7. Los datos pueden exportarse y recuperarse.
8. La interfaz mantiene el sistema visual en escritorio y móvil.
9. Los secretos del GM no son accesibles a usuarios sin permiso.
10. La aplicación puede ampliarse con nuevos tipos de entidad sin reescribir el núcleo.

23. Brief técnico para Codex

Este documento puede utilizarse como especificación inicial para desarrollar el proyecto con Codex. La recomendación es trabajar por fases, manteniendo el repositorio siempre ejecutable.

Prompt inicial recomendado:

```

Construye una aplicación web profesional llamada "Centralita Pathfinder 2e".
Debe funcionar como una base de datos/wiki reutilizable para múltiples campañas.

REQUISITOS:
- Arquitectura modular y escalable.
- Separar compendio global de datos específicos de campaña.
- Frontend responsive con estética de fantasía oscura: #0D1112, #171B1C, #E8D9B8, #B8933F, #7A1F1F, #2B1A12, #F2EBDD.
- Serif para títulos y sans-serif para contenido.
- Sidebar persistente en escritorio y menú adaptable en móvil.
- CRUD desde la interfaz para dotes, conjuros, objetos, criaturas, NPC, localizaciones, personajes, etc.
- PostgreSQL como base de datos.
- API modular.
- Validación backend y frontend.
- Sistema de relaciones entre entidades.
- Buscador global.
- Historial de cambios.
- Importación/exportación JSON versionada.
- Preparar permisos de GM/jugador.
- No hardcodear contenido en los componentes de UI.
- Crear un design system reutilizable.
  
```

PRIMERA FASE:

1. Crear repositorio y arquitectura.
2. Implementar layout y design system.
3. Implementar autenticación básica.
4. Implementar PostgreSQL y migraciones.
5. Implementar Campaign, Character, NPC, Location y Compendium.
6. Implementar CRUD.
7. Implementar buscador global.
8. Implementar relaciones.
9. Implementar dashboard.
10. Añadir tests básicos.

Antes de añadir funcionalidades avanzadas, asegúrate de que la aplicación compila, arranca y los tests pasan.

24. Consideraciones sobre contenido y propiedad intelectual

La arquitectura debe permitir introducir contenido manualmente y distinguir la fuente de cada entrada. Para una distribución pública/comercial, el proyecto debe revisar qué textos, imágenes, nombres, ilustraciones y datos de reglas se incorporan y bajo qué licencia o autorización. La aplicación puede diseñarse de forma neutral para que el usuario introduzca su propio contenido y para que los módulos de datos sean sustituibles.

25. Visión final

La meta no es construir “una app para una campaña”. Es construir un sistema operativo personal para dirigir campañas: una biblioteca de conocimiento, una wiki enlazada, un gestor de personajes y un conjunto de herramientas de dirección.

La arquitectura propuesta permite comenzar con una aplicación relativamente manejable y hacerla crecer progresivamente sin perder la estructura. El núcleo debe ser la relación entre datos: una criatura pertenece al compendio, un encuentro la utiliza, una sesión registra el resultado, una misión conecta personajes y localizaciones, y la wiki mantiene el contexto narrativo.

Resultado objetivo: una única centralita desde la que el usuario pueda crear, consultar, modificar y reutilizar todo el contenido necesario para cualquier campaña.